Fraud Intelligence AI Assistant

Architecture · Logging · Observability · Metrics · Docker

Capstone Project 2025 · LangGraph · Groq · ChromaDB · Tavily · LangSmith

Technology stack

Layer	Technology
Agent orchestration	LangGraph 0.2+ — StateGraph
LLM inference	Groq API — Llama 3.3 70B + fallback chain
Vector database	ChromaDB — cosine similarity, 384-dim
Embeddings	sentence-transformers all-MiniLM-L6-v2 (local)
Web search	Tavily API — research optimised
API layer	FastAPI — webhook receiver · port 8001
UI	Streamlit — live agent dashboard · port 8501
Guardrails	6-layer input + output safety
Logging	structlog — JSON lines, run_id bound
Metrics	Prometheus + Grafana
Tracing	LangSmith + OpenTelemetry
RAG evaluation	RAGAS — faithfulness, relevance, recall
Deployment	Docker Compose

1. End-to-End Pipeline Architecture

Complete flow from webhook trigger through FastAPI, input guardrails, LangGraph Supervisor, four parallel research agents with RAG pipeline, Report agent, RAGAS evaluation, risk decision, output guardrails, and Streamlit dashboard. Every step is observable via the three-pillar logging stack.

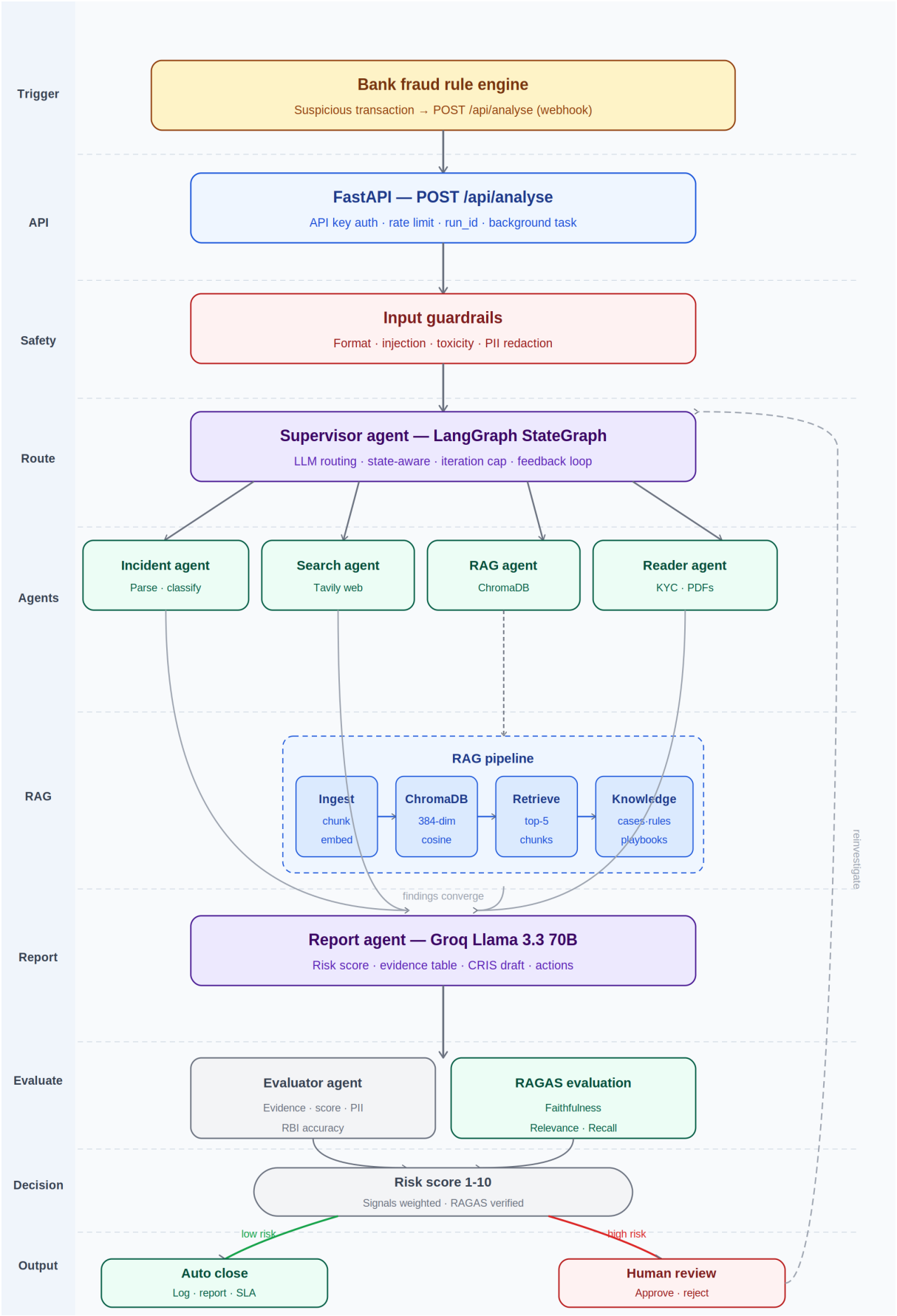

Figure 1 — Full pipeline: webhook trigger → Streamlit dashboard

Pipeline layer descriptions

Layer	Description
Trigger	Bank rule engine fires POST /api/analyse the moment suspicious transaction detected
FastAPI	API key auth · rate limiting · run_id assigned · pipeline started as background task
Input guardrails	Format · injection detection · toxicity · PII auto-redaction — runs before any LLM call
Supervisor	LLM reads state at each iteration · routes to correct agent · handles evaluator feedback
Incident agent	Parses and classifies alert · logs to PostgreSQL/CSV incident database
Search agent	Tavily API — live web: fraud patterns, RBI circulars, SWIFT advisories
RAG agent	ChromaDB cosine search — top-5 past cases, playbooks, RBI guidelines retrieved
Reader agent	PyMuPDF — extracts text from uploaded KYC PDFs and account statements
Report agent	Groq Llama 3.3 70B — synthesises all context into structured fraud report
Evaluator	Evidence grounding · score proportionality · PII check · RBI deadline accuracy
RAGAS	Faithfulness · Answer relevance · Context recall — stored in SQLite per run
Risk decision	Score ≥ 7 → human review · Score < 7 → auto close · Reject → reinvestigate
Output guardrails	PII scan · hallucination detection · citation audit on final report
Dashboard	Streamlit — report, risk score, action buttons, monitoring panel, download

2. Logging · Observability · Tracing · Metrics

Every agent call passes through the @track_agent decorator which emits to three pillars simultaneously — structlog, Prometheus, and LangSmith. Zero monitoring boilerplate exists inside agent code. The decorator handles timing, token recording, error catching, and all three pillar emissions automatically.

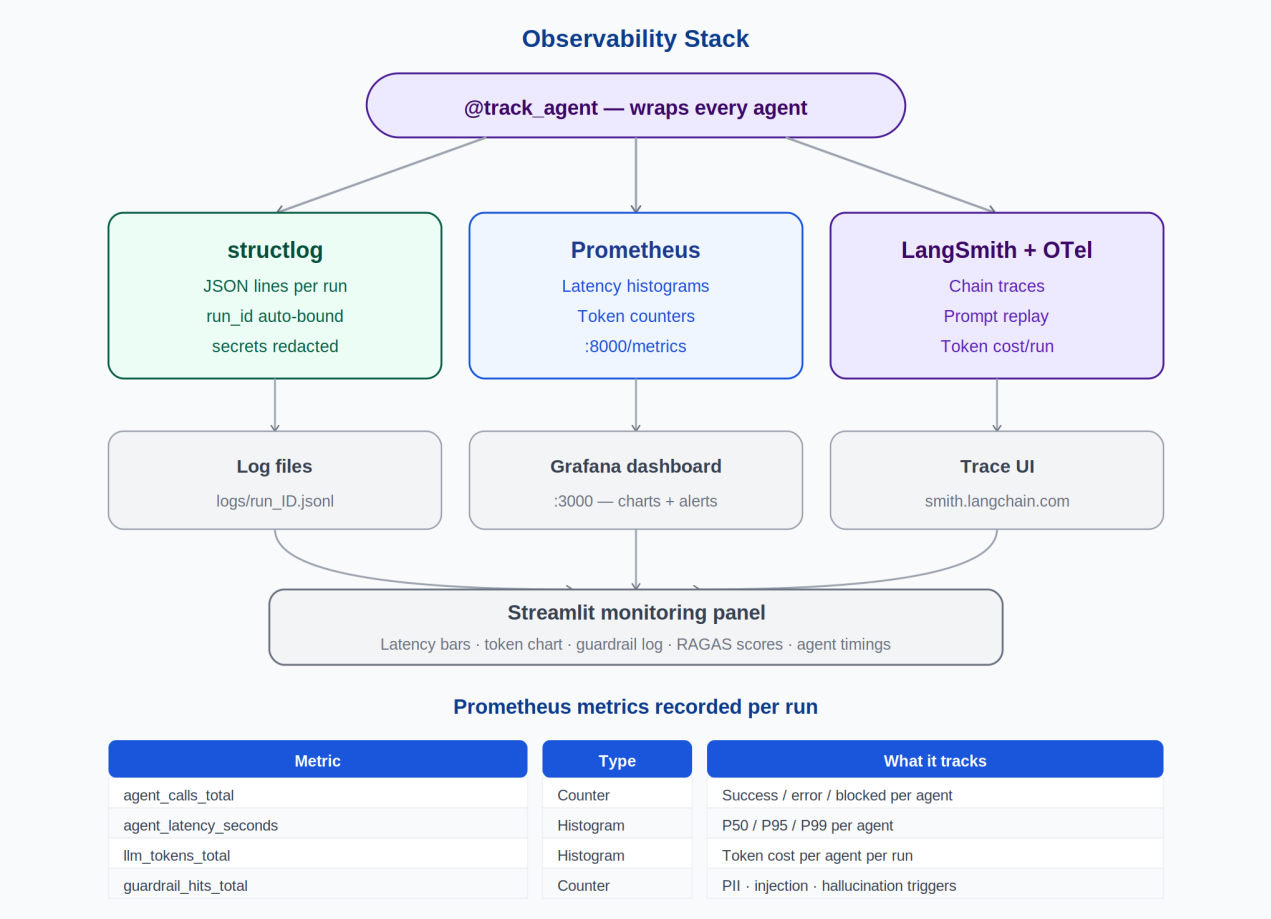

Figure 2 — Three-pillar observability stack with Prometheus metrics table

Structlog — structured logging

Feature	Detail
JSON lines	Every event recorded as structured JSON — machine-parseable by Loki, Datadog, CloudWatch
run_id binding	run_id bound via contextvars — every log line carries the run ID automatically
Secret redaction	API keys, tokens auto-redacted before writing — no secrets in log files
Per-run files	logs/run_a1b2c3d4.jsonl per pipeline run + logs/insightflow.jsonl global
Log levels	INFO: normal flow · WARNING: guardrail hits · ERROR: agent failures

LangSmith — chain tracing

Setup (3 env vars only): LANGCHAIN_TRACING_V2=true · LANGCHAIN_API_KEY=ls_xxxx · LANGCHAIN_PROJECT=fiaa-fraud

LangSmith feature	What you can see
Run timeline	Full LangGraph chain for every run — which agent ran when and how long
Prompt inspect	Exact prompt sent to every LLM call — what context the Report Agent received
Response inspect	Exact LLM output — spot where the report went wrong on a failed run
Token cost	Token count and estimated cost per agent per run
Run replay	Replay any historical run — useful for debugging evaluator rejections

Feedback	Mark runs as good/bad — build a quality dataset over time

Grafana alert rules

Alert rule	Condition	Severity
High writer latency	P95 agent_latency_seconds{agent=report_agent} > 30s	Warning
Injection spike	rate(guardrail_hits_total{type=injection}[1m]) > 5	Critical
High error rate	rate(agent_calls_total{status=error}[5m]) / rate(agent_calls_total[5m]) > 0.1	Critical
Too many active runs	active_runs > 10	Warning
Low RAGAS faithfulness	ragas_faithfulness < 0.7 for 3 consecutive runs	Warning

3. Docker Compose Deployment

All four services — app, ChromaDB, Prometheus, and Grafana — are defined in a single docker-compose.yml. One command starts the complete system. Volumes persist data between container restarts. External APIs (Groq, Tavily, LangSmith) require only API keys in the .env file.

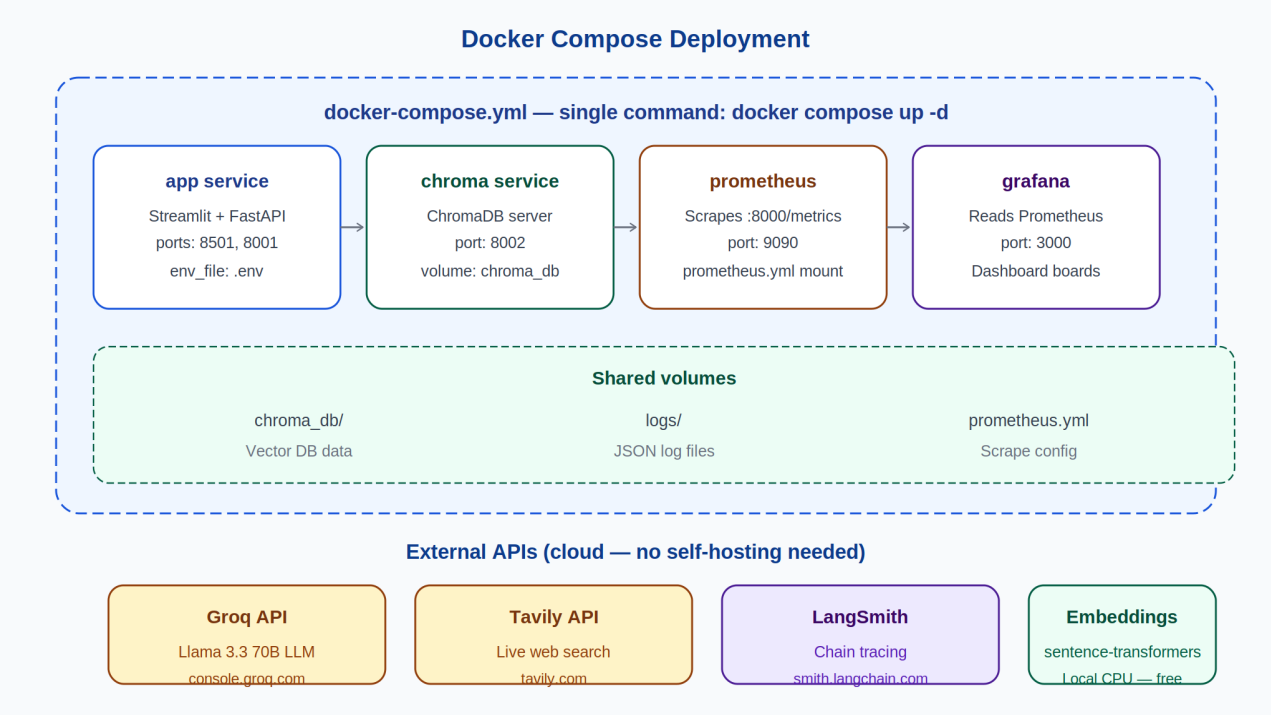

Figure 3 — Four-service Docker Compose deployment

docker-compose.yml — key sections

Service	Image	Ports	Volumes / config
app	Custom Dockerfile	8501 (Streamlit), 8001 (FastAPI)	chroma_db:/ logs:/ env_file:.env
chroma	chromadb/chroma	8002	chroma_db:/chroma/.chroma
prometheus	prom/prometheus	9090	prometheus.yml:/etc/prometheus/prometheus.yml
grafana	grafana/grafana	3000	GF_SECURITY_ADMIN_PASSWORD in environment

prometheus.yml — scrape config

Setting	Value
scrape_interval	5s — scrape every 5 seconds
job_name	insightflow
targets	host.docker.internal:8000 — the metrics endpoint exposed by the app service

One-command startup sequence

Step	Command	Result
1. Configure	cp .env.example .env (edit API keys)	GROQ_API_KEY · TAVILY_API_KEY · LANGCHAIN_API_KEY set
2. Build RAG KB	docker compose run app python -m rag.ingestor	847 chunks indexed into ChromaDB volume
3. Launch all	docker compose up -d	All 4 services running in background
4. Open UI	http://localhost:8501	Streamlit dashboard live
5. Test webhook	curl -X POST localhost:8001/api/analyse ...	Pipeline fires, report appears on dashboard
6. View metrics	http://localhost:3000 (admin/admin)	Grafana dashboard with all metric charts
7. View traces	smith.langchain.com → fiaa-fraud project	Full LangGraph chain trace for every run

Production note: For Windows Server production deployment replace Docker with nssm Windows service for Streamlit and nginx as reverse proxy on port 80. PostgreSQL replaces SQLite. Prometheus and Grafana remain the same.